Лабораторная работа
Параллельное умножение матрицы на вектор с использованием OpenMP

Студент группы ИА332 Рубцов Роман Кириллович

Новосибирск, 2026

1 Цель работы

Реализация и сравнительный анализ эффективности трех алгоритмов параллельного умножения матрицы размера $M \times N$ на вектор с использованием технологии OpenMP. Исследование влияния декомпозиции данных (по строкам, по столбцам, блочная) на производительность и масштабируемость программы при различном количестве потоков выполнения.

2 Конфигурация

- Процессор (CPU): Intel Core i5-10400F (базовая частота 2.90 ГГц);
- Конфигурация ядер: 6 физических ядер / 12 логических потоков;
- Кэш-память:
  - Кэш первого уровня (L1): суммарно 384 КБ (по 64 КБ на ядро: 32 КБ для данных + 32 КБ для инструкций);
  - Кэш второго уровня (L2): суммарно 1.5 МБ (по 256 КБ на ядро);
  - Кэш третьего уровня (L3): 12 МБ общего кэша Intel Smart Cache.
- Оперативная память (RAM): DDR4 16 ГБ;
- Операционная система: Arch Linux (x86_64);
- Компилятор: GCC с флагами оптимизации -O3 и поддержкой OpenMP (-fopenmp).

3 Теоретические сведения

3.1 Операция умножения матрицы на вектор

Результатом умножения матрицы A размером $M \times N$ на вектор x длиной N является вектор y длиной M, элементы которого вычисляются по формуле:

$$y_i = \sum_{j=0}^{N-1} A_{ij} \cdot x_j, \quad i = 0, \ldots, M-1. \tag{1}$$

Вычислительная сложность операции составляет $O(M \cdot N)$. При больших размерах матриц данная операция становится узким местом, требующим параллелизации.

3.2 Стратегии декомпозиции и параллелизации

Для распараллеливания вычислений используются различные стратегии распределения итераций между потоками:

1. **Ленточная декомпозиция по строкам (Row-wise).** Внешний цикл по строкам матрицы распараллеливается директивой `#pragma omp parallel for`. Каждый поток вычисляет скалярное произведение одной или нескольких строк на вектор x. Поскольку каждая строка записывается в независимый элемент результиру vectora $y[i]$, конфликтов записи не возникает. Доступ к элементам матрицы происходит последовательно по памяти (row-major order), что обеспечивает высокую эффективность использования кэш-памяти.

2. **Ленточная декомпозиция по столбцам (Column-wise).** Распараллеливается цикл по столбцам. Каждый поток обрабатывает свой набор столбцов, добавляя вклад $A_{ij} \cdot x_j$ во все элементы результирующего вектора y. Для корректного суммирования вкладов от разных потоков в одни и те же ячейки памяти используется конструкция `reduction(+ : res[0:rows])`. Недостатком данного подхода является нерегулярный доступ к памяти матрицы (прыжки через N элементов), что приводит к промахам кэша.

3. **Блочная декомпозиция (Block-wise).** Матрица разбивается на блоки размером $B \times B$ (в работе использован блок 8×8). Обработка ведется блоками: внешний цикл по блокам строк распараллелен. Внутри блока доступ к элементам матрицы и вектора происходит локально, что улучшает пространственную локальность данных и утилизацию кэш-памяти L1/L2. Данный подход сочетает преимущества параллелизма по строкам с оптимизацией доступа к памяти.

4 Постановка задачи

Необходимо реализовать программу умножения матрицы размером 10000×15000 на вектор тремя способами: по строкам, по столбцам и блочным методом (размер блока 8). Элементы матрицы и вектора заполняются случайными значениями из диапазона $[-10; 10]$.

Требуется провести экспериментальный анализ времени выполнения каждого алгоритма при числе потоков: 1, 2, 4, 6, 8, 12, 16, 32, 64. Время измеряется с помощью `std::chrono::steady_cloc` в миллисекундах. Сравнение эффективности проводится на основе построенных графиков зависимости времени от числа потоков.

5 Результаты выполнения

5.1 Программная реализация

Программа написана на языке C++ с использованием библиотеки OpenMP.

- Функция `dot_rows` использует `#pragma omp parallel for` для внешнего цикла по строкам. Локальная переменная `sum` используется для накопления суммы внутри потока перед записью в результат, что исключает ложное разделение данных.
- Функция `dot_cols` использует `#pragma omp parallel for reduction(+ : res_ptr[0 : rows])` для безопасного суммирования вкладов столбцов в общий вектор результатов.
- Функция `dot_blocks` реализует четырехуровневый цикл обхода блоков. Параллелизм введен по внешнему циклу блоков строк (`bi`). Размер блока выбран равным 8.

5.2 Экспериментальные результаты

Результаты измерений времени выполнения представлены в таблице 1 и на рисунке 1.

Таблица 1: Время выполнения алгоритмов умножения матрицы на вектор (мс)

Потоки	По строкам (мс)	По столбцам (мс)	Блочный 8x8 (мс)
1	152	455	156
2	76	234	82
4	39	123	42
6	27	107	28
8	22	135	36
12	19	132	28
16	22	140	31
32	21	154	32
64	21	153	27

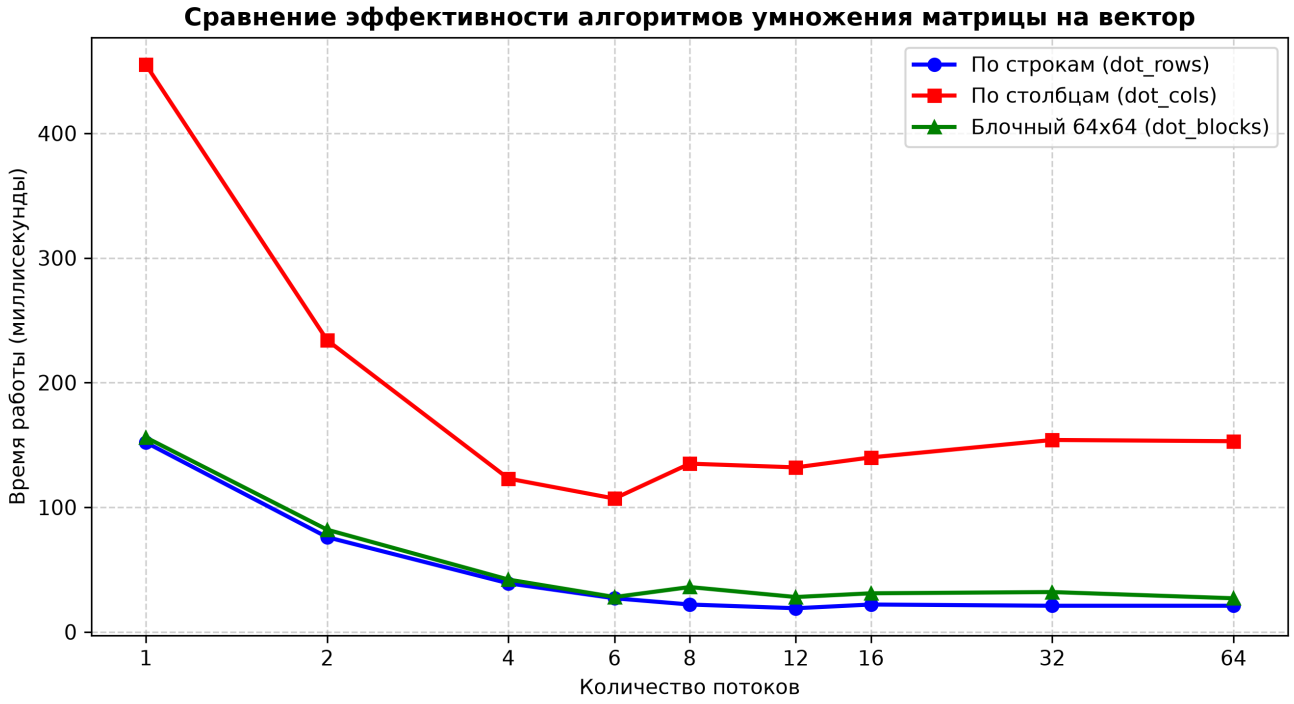


Рис. 1: Сравнение эффективности алгоритмов умножения матрицы на вектор

а

5.3 Анализ эффективности алгоритмов

На основе полученных данных можно выделить следующие закономерности:

- **Алгоритм по столбцам (dot_cols):** Демонстрирует наихудшую производительность. При 1 потоке он работает почти в 3 раза медленнее остальных (455 мс против 150 мс).

Это объясняется двумя факторами: во-первых, доступ к матрице идет по столбцам, что нарушает принцип локальности данных (cache miss при каждом обращении к новой строке); во-вторых, механизм `reduction` для массива создает значительные накладные расходы на синхронизацию и объединение частичных результатов. Рост числа потоков до 6 дает улучшение, но далее наблюдается стагнация и даже деградация (рост времени с 107 мс до 154 мс при 32 потоках).

- **Алгоритм по строкам (dot_rows):** Показывает отличное масштабирование в диапазоне 1–6 потоков (линейное ускорение: 152 → 76 → 39 → 27 мс). Однако после 6 потоков (что соответствует числу физических ядер CPU) ускорение останавливается. При 8, 12, 16 и более потоках время стабилизируется на уровне 20-22 мс. Использование гиперпоточности (SMT) не дает выигрыша, так как задача ограничена пропускной способностью памяти (memory-bound), а не вычислительными ресурсами ядер.

- **Блочный алгоритм (dot_blocks):** На одиночном потоке работает сопоставимо со строковым методом (156 мс). При увеличении числа потоков демонстрирует поведение, схожее со строковым методом, но с небольшим преимуществом (27-32 мс против 21-22 мс у строк на высоких нагрузках, хотя разница находится в пределах погрешности измерений). Блочная структура теоретически должна давать лучший результат за счет кэша, но при размере блока 8 и матрице 10000×15000 эффект не виден та как, данные при построчном умноежении все равно не помещаются полностью в L3 кэш, и основным лимитирующим фактором остается скорость оперативной памяти.

Максимальное ускорение для лучшего алгоритма (по строкам) составляет:

$$S_{max} = \frac{T_1}{T_6} = \frac{152}{27} \approx 5.63. \tag{2}$$

Дальнейшее увеличение потоков сверх числа физических ядер (6) неэффективно.

6 Выводы

В ходе выполнения лабораторной работы были получены следующие результаты:

1. **Сравнение методов декомпозиции:** Ленточная декомпозиция по строкам оказалась наиболее эффективным и простым в реализации подходом для операции умножения матрицы на вектор в формате row-major. Декомпозиция по столбцам показала крайне низкую эффективность из-за плохой локальности данных и накладных расходов на редукцию.

2. **Эффективность блочного метода:** Блочная декомпозиция (8x8) показала результаты, сопоставимые с построчным методом. Это подтверждает, что для данной размерно-

сти матрицы и архитектуры процессора основным «бутылочным горлышком» является пропускная способность памяти, а не организация вычислений внутри кэша.

3. **Масштабируемость:** Эффективное ускорение наблюдается только до количества физических ядер (6 для i5-10400F). Использование 12, 32 и 64 потоков не приводит к снижению времени выполнения, а для метода по столбцам вызывает деградацию производительности.